Reviewer Pack — Coxeter Group Action Patterns and Communication Complexity f…

Entry 83ac1cf84229 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 07:51:02 UTC

Coxeter Group Action Patterns and Communication Complexity for k-Clique

Entry ID: `83ac1cf84229` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 07:51:02 UTC

1. Conjecture

Field A (mathematical branch): Coxeter Group Theory **Field B** (complexity object): Communication Complexity: k-Clique

Statement:

> ['For any given n-vertex graph G with m edges, the communication complexity of determining if G contains a clique of size k is upper bounded by O($k^{(2/3)n}$(1/3)) if and only if the vertex set of G can be partitioned into subsets whose action on each other forms a Coxeter group.', 'For all graphs G, E[X(communication_complexity(G))] ≤ Θ((k^(2/3) + $m^{(4/9))n}$(1/3)) where X is the random variable representing the communication complexity of G.', 'The minimal rank of the action patterns in the associated permutation group for any graph G with a clique of size k is at most $k^{(2/3)n}$(1/3).']

Rationale (proposer's reasoning):

> ['Coxeter groups have been used to classify and study symmetries, which may provide insights into the structure of graphs and their communication complexity.', 'The conjecture suggests a deep connection between group theory and computational tasks, potentially revealing new principles in communication complexity.', 'If true, it could lead to more efficient algorithms for solving problems related to k-clique.']

Taxonomy category: `coxeter_group_action_patterns` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2f32f3f662450535`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The communication complexity of determining a k-clique in an n-vertex graph with m edges is considered supported if it is upper bounded by O($k^{(2/3)n}$(1/3)) and falsified if this bound is exceeded.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter Group Theory" AND "communication complexity" AND k-clique" - "permutation group action patterns" AND Coxeter group AND communication complexity" - "graph clique detection" AND upper bound ON communication complexity AND Coxeter group

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def matrix_multiply(A, B):
    m, n = len(A), len(B[0])
    p = len(B)
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    m, n = len(A), len(A[0])
    Augmented = [A[i] + [b[i]] for i in range(m)]
    for i in range(n):
        max_row = i
        for j in range(i+1, m):
            if abs(Augmented[j][i]) > abs(Augmented[max_row][i]):
                max_row = j
```

```
            Augmented[i], Augmented[max_row] = Augmented[max_row], Augmented[i]
            pivot = Augmented[i][i]
            for j in range(i, n+1):
                Augmented[i][j] /= pivot
            for j in range(m):
                if j != i:
                    factor = Augmented[j][i]
                    for k in range(i, n+1):
                        Augmented[j][k] -= factor * Augmented[i][k]
        return [row[-1] for row in Augmented]

    def is_coxeter_group(G):
        m = len(G)
        if m != len(set(len(row) for row in G)):
            return False
        for i in range(m):
            for j in range(i+1, m):
                if G[i][j] == 0 and G[j][i] == 0:
                    return False
        return True

    def communication_complexity(G, k):
        n = len(G)
        # Placeholder function to compute communication complexity
        # This is a dummy implementation for the sake of testing
        return k * (n ** (1/3))

    def run_trial(seed: int) -> dict:
        random.seed(seed)
        n = random.randint(5, 40)
        m = random.randint(0, n*(n-1)//2)
        G = [[random.choice([0, 1]) if i != j else 0 for j in range(n)] for i in range(n)]
        k = random.randint(3, min(k, n))

        communication_comp = communication_complexity(G, k)
        is_coxeter = is_coxeter_group(G)

        metric_value = communication_comp
        conjecture_holds = False
        counterexample = ""

        if is_coxeter:
            upper_bound = k**(2/3) * n**(1/3)
            if communication_comp <= upper_bound:
                conjecture_holds = True
            else:
                counterexample = "Coxeter group condition violated"

        return {
            "metric_name": "communication_complexity",
            "metric_value": metric_value,
            "instances_tested": 1,
            "conjecture_holds": conjecture_holds,
            "counterexample": counterexample
        }
```

```python
if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    total_metric_value = Fraction(0)
    num_trials = len(seeds)

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        total_metric_value += Fraction(result["metric_value"])
        if not result["conjecture_holds"]:
            counterexample = result["counterexample"]
            break

    mean_metric_value = total_metric_value / num_trials
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / num_trials

    if counterexample:
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seeds[re
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean=<{mean_metric_value}> std=<not_computed> support_fraction=<
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction=<{support_fraction}>")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dbef86bb.py", line 45, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dbef86bb.py", line 27, in run_trial
    k = random.randint(3, min(k, n))
                               ^
UnboundLocalError: cannot access local variable 'k' where it is not associated with a value
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying whether the communication complexity of determining a k-clique in an n-vertex graph with m edges is upper bounded by $O(k^{(2/3)n}(1/3))$. | next: Re-run the test code to ensure it completes without crashing and produces results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11752
2	preregistration	ollama_remote	glm4:latest	0	0	5503
3	novelty	ollama_remote	glm4:latest	0	0	4677
4	novelty	ollama_remote	glm4:latest	0	0	5593
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17065
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9821
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	5866
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11884
9	judge	ollama_remote	glm4:latest	0	0	11148

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 83309 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/83ac1cf84229.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/83ac1cf84229.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/83ac1cf84229.tar.gz` (if generated)